

EE200: Signals, Systems and Networks

Course Project Report

Q3A: Sonic Signatures — Magical Mystery Tune [7.5%]	Q3B: Signals to Softwares — Zapptain America [5%]
--	--

Instructor: Dr. Tushar Sandhan
Department of Electrical Engineering, IIT Kanpur

Marks: Q3A – 7.5% | Q3B – 5%

Table of Contents

- 1. Introduction & Motivation
- 2. Why a Global DFT is Insufficient
- 3. Spectrogram: Time-Frequency Analysis
- 4. Effect of Window Length on Resolution
- 5. Constellation Map: Peak Picking
- 6. Hashing: Paired Peaks vs. Single Peaks
- 7. Indexing the Song Database
- 8. Identification: Offset Histogram
- 9. Robustness: Noise Tolerance
- 10. Robustness: Pitch Shift
- 11. Q3B – Zapptain America App
- 12. Conclusions

1. Introduction & Motivation

Automatic music identification is one of the most elegant applications of digital signal processing. A system like Shazam can identify a song from a noisy few-second clip against a database of millions of tracks in milliseconds. The key insight is that it never compares raw audio waveforms — instead it converts each audio clip into a compact, robust **fingerprint** derived from its time-frequency representation.

In this project we implement a complete music identifier from first principles, covering:

- Computing the Short-Time Fourier Transform (STFT) / spectrogram.
- Extracting a constellation map of local spectral peaks.
- Building paired-peak hashes and storing them in an indexed database.
- Matching a query clip by aligning offset histograms.
- Analysing robustness to additive noise and pitch shifts.
- Wrapping the identifier in a deployable Streamlit web application (Q3B).

2. Why a Global DFT is Insufficient

The Discrete Fourier Transform of a length- N signal $x[n]$ is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$

A plot of $|X[k]|$ shows *which* frequencies are present across the entire song but gives zero information about *when* they occur. Two scenarios — a steady tone and a brief melodic burst at the same frequency — are indistinguishable in the global DFT. Since the timing of note onsets and melodic patterns is essential for identifying a song, a single transform of the whole recording is unsuitable as a fingerprint.

3. Spectrogram: Time-Frequency Analysis

A **spectrogram** resolves the time-frequency trade-off by applying the DFT locally: a short analysis window $w[n]$ is slid along the signal in steps of H samples (hop length), and the DFT is computed for each windowed excerpt. Stacking the resulting magnitude spectra as columns produces a 2-D image with time on the horizontal axis and frequency on the vertical axis.

Formally the Short-Time Fourier Transform (STFT) is:

$$X[m, k] = \sum_n x[n] w[n - mH] e^{-j2\pi kn/N}$$

We use a **Hann window** to minimise spectral leakage, and the spectrogram magnitude $S[m, k] = |X[m, k]|$ is displayed in decibels.

Parameters used in this implementation:

Parameter	Value	Rationale
Sample rate (fs)	22 050 Hz	Standard CD-quality for music

FFT size (N)	2048 samples	~93 ms; good frequency resolution (10.8 Hz/bin)
Hop length (H)	512 samples	75% overlap; smooth time axis (~23 ms steps)
Window	Hann	Excellent sidelobe suppression; minimal leakage

4. Effect of Window Length on Time vs. Frequency Resolution

The fundamental uncertainty principle of signal processing states that time and frequency resolution cannot simultaneously be made arbitrarily small:

$$\Delta t \cdot \Delta f \geq 1$$

Three window configurations were tested:

Config	N _{FFT}	Time res.	Freq res.	Observation
Short	256	~12 ms	86 Hz/bin	Transients visible; harmonic lines blurred
Medium	2048	~93 ms	10.8 Hz/bin	Practical balance; chosen for fingerprinting
Long	8192	~371 ms	2.7 Hz/bin	Crisp harmonics; transient onsets smeared

(Green row = chosen configuration)

Key insight: For fingerprinting, the medium window (N=2048) is optimal. A short window misses fine harmonic structure (peaks are spectrally blurred and will collide across songs). A long window smears onset timing, making the constellation map too sparse in time.

5. Constellation Map: Peak Picking

From the spectrogram we retain only the **local maxima** — points whose magnitude is higher than all neighbours in a (NEIGHBOR x NEIGHBOR) = (20 x 20) time-frequency window AND exceeds the median by at least 10 dB. This yields a sparse "constellation" of typically a few hundred peaks per minute of audio.

Algorithm:

- Compute $S_{dB}[m, k] = 20 \log_{10}(|X[m, k]| + \epsilon)$.
- Apply 2-D maximum filter with a 20x20 footprint.
- Declare a peak wherever S_{dB} equals its local maximum and $S_{dB} > \text{median}(S_{dB}) + 10 \text{ dB}$.
- Discard the lowest 5 frequency bins (DC and near-DC noise).

Why sparsity matters: A dense map would generate billions of hash collisions across songs and make matching ambiguous. The constellation retains only the most energetically prominent events — typically the fundamental frequency, strong harmonics, and percussive transients — which are reproducible even in noisy conditions.

6. Hashing: Paired Peaks vs. Single Peaks

Each **anchor peak** at (t_1, f_1) is paired with up to $FAN_VALUE = 15$ nearby **target peaks** (t_2, f_2) occurring within $MAX_DELTA_T = 200$ time frames. Each pair is encoded into a 32-bit hash:

```
hash = (f1 & 0x1FF) | ((f2 & 0x1FF) << 9) | ((Δt & 0x3FF) << 18)
```

The hash is stored together with the anchor time offset t_1 .

Single Peaks vs. Paired Hashes Comparison:

Property	Single peak (f, t)	Paired hash (f1, f2, Δt)
Information per entry	1 frequency	2 frequencies + time gap
Collision probability	High — common notes shared across songs	Very low — triplet is song-specific
Offset histogram	Broad, ambiguous peak	Sharp, decisive spike at true offset
Noise robustness	Poor	Good
Database size	Small	~15x larger but tractable

Why pairing is decisive: The probability that two different songs share the same $(f1, f2, \Delta t)$ triplet is orders of magnitude lower than sharing a single frequency bin. When we histogram the time offset $\delta = t_{DB} - t_{query}$ for all matching hashes, a correct match produces all its votes at a single offset (the song's temporal alignment), forming a sharp spike. A wrong song produces only scattered, random collisions with no dominant offset.

7. Indexing the Song Database

For each song in the database:

- Load audio at 22 050 Hz, mono.
- Compute the magnitude spectrogram ($N=2048$, $H=512$, Hann window).
- Extract the constellation map.
- Generate all paired hashes.
- Store each $(hash_value) \rightarrow (song_id, anchor_time_offset)$ in a Python dict.

The resulting lookup table maps any hash value to the list of songs that contain it and at what temporal position. This pre-computation is done once at startup; subsequent queries are answered in milliseconds.

8. Identification via Offset Histogram Matching

Given a query audio clip:

- Extract its constellation map and generate hashes.
- For each query hash (h, t_q) , look up h in the database.
- For each database entry $(song_id, t_{db})$, compute $\delta = t_{db} - t_q$.
- Accumulate votes: $votes[song_id][\delta] += 1$.
- The best match is the $song_id$ whose maximum vote count is highest.

A genuine match produces votes that all align at a single offset δ^* (the temporal position of the query within the database song). Wrong songs produce only low-level noise in the histogram — no dominant offset emerges.

9. Robustness: Additive White Gaussian Noise (AWGN)

White Gaussian noise at SNR dB is added to the query signal:

$$\sigma_{\text{noise}}^2 = P_{\text{signal}} / 10^{\text{SNR}/10}$$

SNR (dB)	Match correct?	Explanation
30	Yes	Noise barely perceptible; constellation unchanged
20	Yes	Background noise level; strong peaks still dominant
10	Yes	Noisy environment; key peaks survive
5	Yes (marginal)	Near the limit; some peaks corrupted
0	No	Noise power = signal power; constellation destroyed
-5	No	Noise dominant; hash matches are random

Why pair hashes help with noise: A noise-corrupted signal may have many spurious peaks. However, for a spurious peak pair to produce a *matching* hash it must precisely replicate two specific frequency bins and their time gap — a vanishingly unlikely event. Only the true signal peaks survive as consistent matches.

10. Robustness: Pitch Shift

The query is pitch-shifted by `librosa.effects.pitch_shift()` before matching:

Shift (semitones)	Match correct?
-2	No
-1	No
0 (original)	Yes
+1	No
+2	No

Why pitch shifts defeat the identifier: Our hashes encode *absolute* frequency bins (f_1, f_2). A uniform pitch shift multiplies all frequencies by a constant factor $2^{s/12}$ (where s is the semitone shift). Every peak moves to a different bin, so every hash value changes and no entry in the database matches. The song sounds nearly identical to a human listener (the JND for pitch is roughly a quarter of a semitone), yet the fingerprint is completely different.

Proposed fix: Encode the *ratio* f_2/f_1 (quantised to a small number of bits) instead of absolute bin indices. Frequency ratios are invariant to uniform pitch scaling, making the hash pitch-robust.

11. Q3B – Zaptain America: Streamlit Web Application

The full identifier is wrapped in a Streamlit app (app_q3b.py) providing two modes:

Single-clip mode:

- User uploads a query audio file (mp3/wav/flac/ogg/m4a).
- App computes and displays: spectrogram, constellation map, offset histogram.
- Matched song name is shown prominently.

Batch mode:

- User uploads multiple query clips simultaneously.
- App identifies all clips and presents a table.
- User downloads results.csv with columns: filename, prediction.

Deployment instructions:

- Push app_q3b.py, song_database/, requirements.txt to a GitHub repository.
- Navigate to <https://share.streamlit.io> and connect the repository.
- Set the main file as app_q3b.py and deploy.
- The indexed database ships with the app so it works immediately on launch.

requirements.txt:

```
streamlit>=1.35 numpy scipy matplotlib librosa soundfile
```

12. Conclusions

We have successfully implemented a Shazam-like audio fingerprinting system that demonstrates the power of spectral analysis in practical signal processing:

- The global DFT is insufficient for identification because it discards temporal information.
- The STFT/spectrogram restores timing by applying localized Fourier analysis.
- Window length governs the time-frequency resolution trade-off: $N=2048$ samples is optimal for music fingerprinting.
- Peak picking yields a sparse, robust constellation map.
- Paired-peak hashing (encoding f_1 , f_2 , Δt) is far more discriminative than single-peak matching; it produces a sharp offset histogram spike for true matches.
- The system tolerates moderate additive noise (~ 10 dB SNR) but fails for pitch shifts of even 1 semitone — a known limitation of absolute-frequency hashing.
- The Streamlit app (Q3B) wraps the identifier for both interactive single-clip use and automated batch evaluation with results.csv output.

End of Report